

TestSplit Functional Specification

Authors: Samson Oloruntola (22714745) & Marjia Siddik (22306501)

Supervisor: Dr Paul Clarke

Module: CSC1097 - Final Year Project

Submission Date: 28/11/2025

Table of Contents

1. Introduction.....	3
1.1 Overview.....	3
1.2 Business / Academic Context.....	3
1.3 Glossary.....	4
2. General Description.....	5
2.1 Product Perspective.....	5
2.2 User Characteristics & Personas.....	6
2.3 Operational Scenarios.....	8
2.4 Constraints.....	11
3. Functional Requirements (FRs).....	12
3.1 Requirement 1 (FR1): Parse Test Results from Standard Formats.....	12
3.2 Requirement 2 (FR2): Test Profiling Engine.....	12
3.3 Requirement 3 (FR3): Parallelisation and Test Grouping.....	13
3.4 Requirement 4 (FR4): CI Configuration Generation.....	16
3.5 Requirement 5 (FR5): Local Data Storage.....	16
3.6 Requirement 6 (FR6): Command-Line Interface (CLI).....	17
3.7 Requirement 7 (FR7): Local Dashboard for Historical Metrics.....	18
3.8 Requirement 8 (FR8): Containerised Execution Environment.....	18
3.9 Requirement 9 (FR9): Historical Trend Analysis.....	19
4. System Architecture.....	20
4.1 System Modules and Functions.....	20
4.2 Interaction Flow (High-Level).....	23
4.3 Environmental Context.....	25
4.4 Reference Diagrams.....	26
4.4.1 Job Distribution Schema.....	28
4.4.2 Profile Schema.....	29
4.4.3 Historical Data Schema.....	29
4.5 Non-Functional Design Choices.....	31
4.6 Non-Functional Requirements.....	31
4.6.1 Runtime Environment.....	31
4.6.2 Test Suitability Requirements.....	32
4.6.3 Quality Requirements.....	32
4.6.4 Performance Requirements.....	32
5. Testing and Evaluation Plan.....	32
5.1 Testing Goals.....	32
5.2 Test Types and Mapping.....	32
5.3 Evaluation Criteria and Thresholds.....	33
5.4 Test Data.....	33
6. Preliminary Schedule.....	34
7. Appendices.....	35
8. References.....	36

1. Introduction

1.1 Overview

TestSplit is a tool that optimises Continuous Integration (CI) pipelines by analysing independent unit-test performance and generating optimised configuration files that distribute tests across parallel jobs. It integrates into existing development and CI environments with minimal setup and no infrastructure changes.

The tool uses the **Longest Processing Time (LPT)** scheduling algorithm [1], [2], [3], which aims to minimise total runtime by assigning longer tests first, allowing a more even workload across jobs. This process reduces waiting time without requiring changes to the tests themselves. Over time, TestSplit builds a history of test performance, helping teams detect slowdowns, identify regressions, and optimise their pipelines.

Key functions of the tool include:

- Parsing test results from JUnit XML file formats
- Profiling test execution times and storing results in a central database
- Generating optimised CI configuration files for GitHub Actions and GitLab CI
- Providing a command-line interface for profiling and configuration generation
- Displaying historical trends and performance metrics through a web dashboard

TestSplit integrates into CI workflows by generating configuration files that assign test groups to parallel jobs. It works with standard test output formats and requires no modifications to the test code or existing CI pipelines.

1.2 Business / Academic Context

This project is not sponsored by a specific business organisation, but it addresses a common and well-documented challenge [4], [5], [6] in modern software development workflows: slow, inefficient Continuous Integration (CI) pipelines driven by large, unbalanced test suites. In many organisations, particularly those practising agile development or continuous delivery, long CI wait times can reduce developer productivity, slow feedback loops, and delay deployments.

TestSplit can be deployed to any software development team that relies on automated testing in CI pipelines. This includes startups, mid-size development teams, and large enterprises using CI/CD platforms such as GitHub Actions and GitLab CI, particularly those managing complex monorepos or legacy codebases where test suites have become performance bottlenecks.

By automatically profiling test performance and intelligently distributing tests across parallel jobs, TestSplit offers a practical solution to improve CI efficiency, reduce infrastructure costs, and support faster development cycles.

1.3 Glossary

Name	Description
LPT Algorithm	The Longest Processing Time scheduling algorithm is used to distribute tests across parallel jobs for optimal workload balance.
Baseline Run	The initial profiling execution that establishes reference timings for future comparisons and optimisations.
CI (Continuous Integration)	A software development practice where code changes are automatically built and tested to maintain integration stability.
CI Configuration	YAML-based configuration file that defines job matrices, parallel test runs, and build stages for CI platforms like GitHub Actions and GitLab CI.
CLI (Command-Line Interface)	The text-based interface through which users interact with TestSplit to profile tests and generate CI configurations.
Containerisation	The process of packaging software and dependencies into isolated environments (containers) for consistent execution.
Dashboard	The frontend web interface for visualising profiling results, job distribution balance, and historical trends.
Dataset/Profile Data	Structured performance data (e.g., JSON) generated during profiling for reuse and trend analysis.
Docker	The container platform used by TestSplit to execute profiling tasks in consistent, reproducible environments.
Job/Job Group	A logical set of tests scheduled to run in parallel as part of a CI pipeline, generated by the LPT algorithm.
JUnit	A Java testing framework that produces XML-formatted results; these serve as input data for TestSplit's parser.
LPT Scheduler	Core module implementing the Longest Processing Time algorithm to determine balanced test groupings.
Parser Module	Component responsible for reading and interpreting JUnit XML files to extract test metadata and durations.
Profiling	The process of analysing test execution times to identify bottlenecks and produce data for optimisation.

Regression Detection	The identification of performance degradations or runtime imbalances across multiple profiling runs.
Speed-Up Factor	A calculated metric showing the improvement in CI runtime achieved through optimised test distribution.
Test Distribution	The allocation of test cases into balanced job groups to achieve minimal overall runtime.
TestSplit	The overall system that analyses test performance, balances workloads, and generates an optimised CI configuration
YAML File	Configuration file format used by CI systems to define workflows, runners, and parallel job setups.

2. General Description

2.1 Product Perspective

TestSplit is an open-source tool that optimises Continuous Integration (CI) pipelines by analysing unit-test performance and generating a pipeline configuration that distributes tests across parallel jobs. TestSplit integrates into existing development and CI environments, providing automation without requiring major infrastructure changes.

Local Development:

- Developers use the TestSplit CLI to analyse JUnit test results and create optimisation profiles.
- The CLI parses JUnit 4 and 5 XML output files to extract individual test durations and compute workload distributions using the Longest Processing Time (LPT) scheduling algorithm.
- Profiling runs are executed in a consistent, containerised environment (Docker) with fixed resource limits to ensure reproducible test timings and stable benchmarking [7], [8].
- This approach allows developers to detect slow tests, measure performance improvements, and generate optimised CI configurations for use in their projects.

CI Pipelines:

- TestSplit integrates with platforms such as GitHub Actions and GitLab CI by generating ready-to-use YAML configuration files [9], [10].
- These configurations define how tests are distributed across multiple parallel jobs based on measured execution times and the LPT-derived scheduling strategy.

- Once deployed, the CI platform executes tests in parallel according to the optimised configuration, reducing total pipeline runtime without modifying existing tests.

Dashboard:

- The TestSplit dashboard provides an interactive web interface for visualising profiling results, optimisation metrics, and the job distribution balance.
- It is implemented as a lightweight React application that loads data locally from CLI-generated JSON files, enabling developers to review performance trends and compare baseline versus optimised runs.
- The dashboard supports deployment to static hosting services such as Vercel, Netlify, or GitHub Pages.

2.2 User Characteristics & Personas

The users of TestSplit are primarily technical professionals who work with Continuous Integration (CI) pipelines and automated testing. These include software developers, DevOps engineers, QA engineers, and technical leads who maintain CI workflows and test infrastructure.

User Expertise:

Most users are expected to have a background in computer science or software engineering. They will be familiar with version-control systems and CI/CD tools such as GitHub Actions and GitLab CI that use JUnit testing frameworks.

Users can be grouped into two main categories:

- **Beginner Users:** Junior developers or team members with limited experience in CI optimisation. They may understand how to run automated tests, but have little knowledge of test profiling or performance tuning. These users benefit from simple CLI commands, clear documentation, and visual dashboards with minimal configuration.
- **Advanced Users:** DevOps engineers and senior developers experienced in tuning CI pipelines, analysing test performance, and customising CI configurations. These users seek deeper insight into test distribution and more control over how tests are split and scheduled.

User Objectives:

From the user's perspective, TestSplit is a tool to:

- Reduce CI pipeline duration by distributing tests more effectively across parallel jobs.
- Profile and analyse test execution times to identify slow or unstable tests.
- Generate configuration files that integrate directly with existing CI workflows.
- Compare baseline and optimised test distributions to validate performance improvements.

- Increase team productivity by shortening feedback cycles during development.

Usage Context:

Users interact with TestSplit primarily through a command-line interface (CLI) and a local web dashboard. The CLI parses JUnit test results, profiles execution times, and generates optimised CI configuration files. The dashboard loads locally generated profiling data to visualise performance metrics, test-suite balance, and historical trends between runs.

Both tools operate on the same development machine or within a reproducible containerised environment, enabling consistent results when profiling, optimising, or validating CI pipelines.

TestSplit integrates with existing CI systems, such as GitHub Actions or GitLab CI, without requiring changes to tests or the repository structure.

Desirable Features (Wish List):

In future phases, users may want:

- Support for additional test formats (e.g., Jest).
- CI configuration generation for other systems (e.g., Jenkins, CircleCI).
- Flaky-test detection and alerting.
- Advanced visualisations (e.g., job timelines, test-heatmaps).

These enhancements are outside the scope of the initial implementation.

2.3 Operational Scenarios

Simplified Use Cases:

The following simplified use cases summarise the core functional capabilities of the TestSplit system. Each use case represents a key interaction or outcome that the system provides to its users.

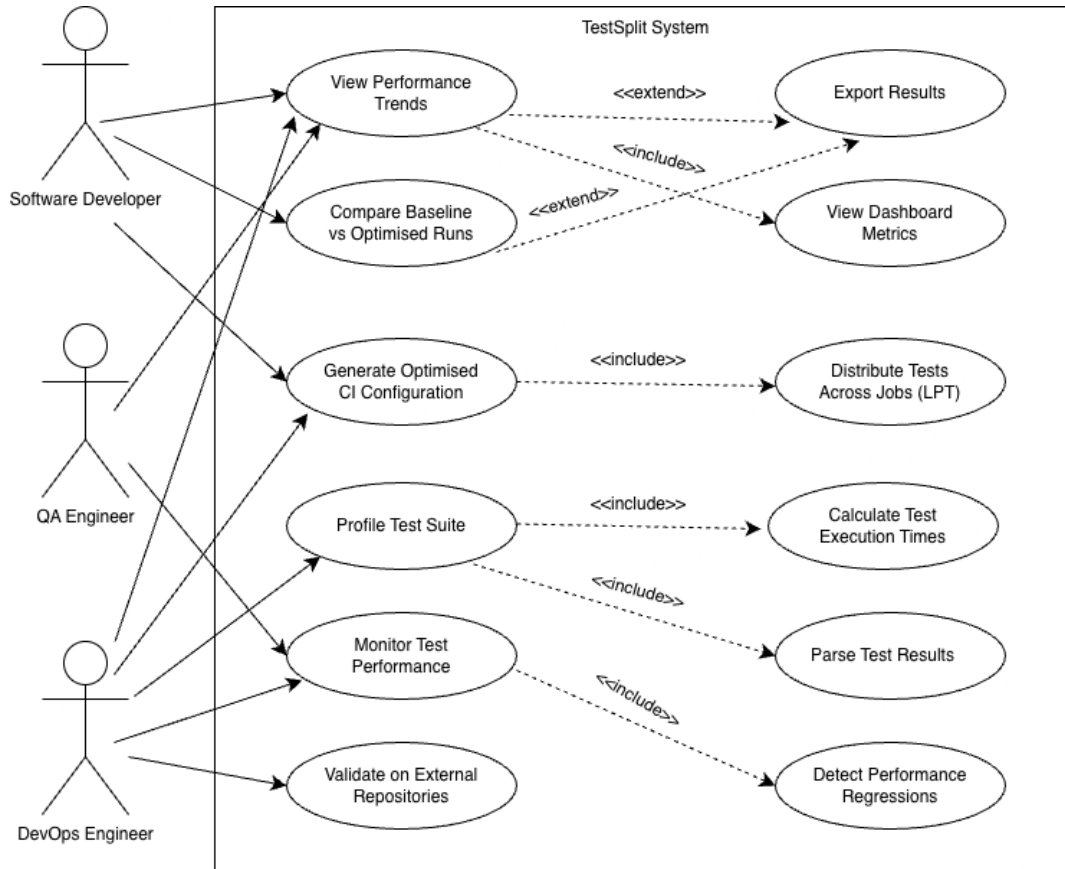


Figure 1: Use Case Diagram

Profile Test Suite:

- Analyse unit-test results to measure individual test durations and identify performance bottlenecks.
- Generate recommendations for balanced parallel distribution based on measured execution times.

Generate Optimised Configuration:

- Automatically generate GitHub Actions or GitLab CI YAML configuration files that define parallel job distribution to improve CI efficiency.

View Performance Trends:

- Load profiling data into the local dashboard to visualise performance improvements and runtime trends across multiple CI runs.

Compare Baseline vs Optimised Runs:

- Evaluate performance before and after applying TestSplit's recommendations by calculating the total runtime reduction achieved in CI pipelines.

Validate on External Repositories:

- Apply TestSplit to open-source projects to assess its effectiveness and consistency in different CI environments.

Export Results:

- Export profiling and analysis data in JSON or CSV formats for reporting, documentation, and research.

Scenario A – Initial Test Optimisation (Software Developer):

User: Alex, a software developer working on a mid-sized open-source project.

Objective: Reduce the duration of the project's unit-test stage in the Continuous Integration (CI) pipeline.

Environment:

- **User Device:** Developer laptop or workstation.
- **System Interface:** TestSplit CLI.
- **Dependencies:** JUnit test framework, GitHub Actions CI.

Steps:

1. Alex runs the project's test suite locally using the existing JUnit framework, which produces results in XML format.
2. Using the CLI, Alex profiles the exported test results to analyse execution times and identify slower tests.
3. TestSplit processes the data and recommends dividing the test suite into four parallel jobs, estimating a potential two to three-fold reduction in total execution time compared with the existing single-job setup.
4. Alex generates an optimised GitHub Actions configuration file using the CLI, which includes the proposed parallel job distribution.
5. The generated configuration file is committed to the repository and pushed to GitHub.
6. On the next CI run, tests execute in parallel using the optimised configuration.
7. The total runtime decreases from approximately 9 minutes to 3.5 minutes, improving feedback time for pull requests and overall productivity.

Scenario B – Performance Review and Regression Detection (DevOps Engineer):

User: Jordan, a DevOps engineer responsible for maintaining CI/CD infrastructure.

Objective: Monitor test performance over multiple profiling runs and detect regressions in test distribution.

Environment:

- **User Device:** Workstation or laptop running Dockerised TestSplit environment.
- **System Interface:** TestSplit CLI and local dashboard.
- **Dependencies:** JUnit test framework, GitHub Actions or GitLab CI.

Steps:

1. Jordan profiles test results across several CI runs using the CLI and stores the generated JSON outputs.
2. The local dashboard loads these profiling JSON files and displays test durations, job balance, and predicted speed-ups.
3. The trend chart shows improvement from an average $2.8\times$ speed-up to $3.4\times$ after recent optimisations.
4. Jordan notices a runtime imbalance, suggesting a newly added test is taking longer than expected.
5. The dashboard highlights the test responsible for the imbalance.
6. After optimising that test, Jordan re-runs TestSplit to verify that the workload distribution and total runtime have stabilised.

This workflow demonstrates TestSplit's ability to identify regressions and verify improvements in a reproducible, static environment.

Scenario C – Flaky Test Detection (Future Enhancement):

User: Priya, a QA engineer responsible for ensuring test reliability.

Objective: Identify and monitor flaky tests that show inconsistent execution times or intermittent failures.

This feature would analyse multiple profiling runs to identify tests with high variance in duration. A stability score would be assigned to each test, helping engineers locate nondeterministic tests that cause unreliable performance. This is outside the scope of the initial implementation but will be explored as a potential Phase B enhancement.

2.4 Constraints

Technical Constraints:

- **Supported Frameworks:** TestSplit supports any testing framework that produces JUnit-compatible XML results.
- **Runtime Environment:** The CLI requires Node.js 18 or later.
- **Containerisation:** Profiling and validation are executed inside a Docker container to maintain a consistent runtime environment across tests and ensure reproducible results.
- **Local Storage Access:** The CLI must be able to read and write local profiling data for comparison between runs.
- **Frontend Compatibility:** The dashboard requires a modern browser with ES6+ support (e.g., Chrome, Firefox, Edge, Safari).

Platform Constraints:

- **CI Integration:** TestSplit generates configurations for GitHub Actions and GitLab CI, both of which support parallel job execution.
- **Parallelisation Requirements:** The target CI platform must allow test sharding or file-pattern-based job distribution.
- **Operating Systems:** The CLI supports macOS (primary development), Linux, and Windows.

Resource Constraints:

- **Development Environment:** A machine with at least 4 CPU cores and 8 GB RAM is recommended.
- **Container Baseline:** Validation runs are executed under a controlled, steady-state Docker configuration to ensure performance consistency and fairness between baseline and optimised runs [11].
- **Network Connectivity:** Internet access is only needed for package installation or CI execution. All profiling and optimisation run locally.

Security and Privacy Constraints:

- All profiling and test metadata remain local to the user's environment.
- No data is transmitted externally or stored on external servers.
- TestSplit does not access or modify source code beyond reading JUnit XML results.

Algorithmic Constraints:

- **Independence Assumption:** The LPT scheduling algorithm assumes that all tests can execute independently, without requiring a dependency ordering.
- **Runtime Stability:** Profiling is performed in a containerised environment to reduce variability from host hardware [11].

- **Historical Data Dependency:** The first profiling run establishes baseline timings used to refine later distributions.
- **Scope of Tests:** TestSplit is designed for individual unit tests that run independently; hence, tests that need to run in a specific order or depend on other tests are not suitable for TestSplit.

3. Functional Requirements (FRs)

3.1 Requirement 1 (FR1): Parse Test Results from Standard Formats

Description:

The system must parse test result files exported from JUnit (versions 4 and 5) [12], [13]. It must extract relevant metadata, including test names, durations, and execution status (passed, failed, or skipped). Parsed data forms the foundation for all subsequent profiling and optimisation processes.

Criticality:

High: This is the first and most essential step in the workflow.

Technical Issues:

- Minor variations in XML structure between JUnit 4 and 5.
- Handling malformed or incomplete result files [14].
- Managing nested or parameterised test cases.

Mitigation Strategies:

- Implement strict XML schema validation with fallback error handling.
- Ignore or flag malformed entries without halting execution.
- Design extensible parser modules to support other test frameworks later (e.g., Jest) [15], [16].

Dependencies:

- This is the first operation in the system pipeline. Its output provides structured input to the profiling engine (FR2).
- Errors or inconsistencies here can cascade into inaccurate profiling or grouping results (FR3).

3.2 Requirement 2 (FR2): Test Profiling Engine

Description:

The system must calculate each test's execution time and store profiling results for reuse [17]. This includes calculating averages across multiple runs and detecting anomalous durations that may indicate instability or environmental issues.

Criticality:

High: Accurate profiling is essential for balanced parallelisation and reliable speed-up measurements.

Technical Issues:

- Inconsistent timing data across hardware or runs.
- Missing or zero-duration entries.
- Handling flaky or intermittent test behaviours.
- Inaccurate profiling data can lead to poor LPT scheduling outcomes or misleading dashboard metrics (FR7/FR9).

Mitigation Strategies:

- Normalise timings using multiple-run averaging or statistical smoothing.
- Flag tests with unstable or zero durations for user review.
- Run profiling inside a containerised environment to ensure reproducibility.

Additional Features:

- Perform extraction analysis on parsed XML data to ensure each test includes name, duration, status, and suite/class grouping.
- Normalise durations to a consistent unit and ensure all durations are numeric.
- Flag zero-duration, missing or malformed duration entries.
- Compute multi-run statistics (mean, variance, σ) for each test.
- Identify timing instability, spikes, or unusually low durations.
- Capture and validate environmental metadata (CPU, OS version, container version, timestamp) against previous runs.

Dependencies:

- Requires structured test data from the parser (FR1).
- Outputs profiling results in a consistent format, consumable by the grouping algorithm (FR3), and stores them locally (FR5).

3.3 Requirement 3 (FR3): Parallelisation and Test Grouping

Description:

The system must distribute tests into balanced parallel jobs using the Longest Processing Time (LPT) scheduling algorithm. The goal is to minimise overall CI runtime by ensuring that each job has a similar total execution duration [18], [19].

Pre-Scheduling Validation

- Validate the presence and numeric correctness of all test durations.
- Sort tests in descending duration order for LPT.

- Aggregate total suite duration from profiler results.

Criticality:

High: This is the core optimisation mechanism that delivers measurable improvements in CI performance.

Post-Scheduling Analysis:

- Compute job-level metrics: maxTime, minTime, idealTime, and balanceRatio.
- Identify outlier jobs with significantly uneven load.
- Compute predicted speed-up using totalDuration and criticalPath (slowest job).

Technical Issues:

- Trade-off between job count and runtime.
- Variation in test durations between runs [20].
- Impact of missing or incomplete profiling data.

Mitigation Strategies:

- Allow users to specify the number of desired parallel jobs.
- Automatically recalculate groupings as new profiling data becomes available.
- Re-run the algorithm periodically to account for regressions or codebase changes.
- Annotate cases where a single long test dominates a group.

Dependencies:

- Requires accurate timing data from the profiling engine (FR2).
- Outputs balanced job groups that are consumed by the CI configuration generator (FR4).
- Relies on consistent historical data stored locally (FR5) for iterative improvement.

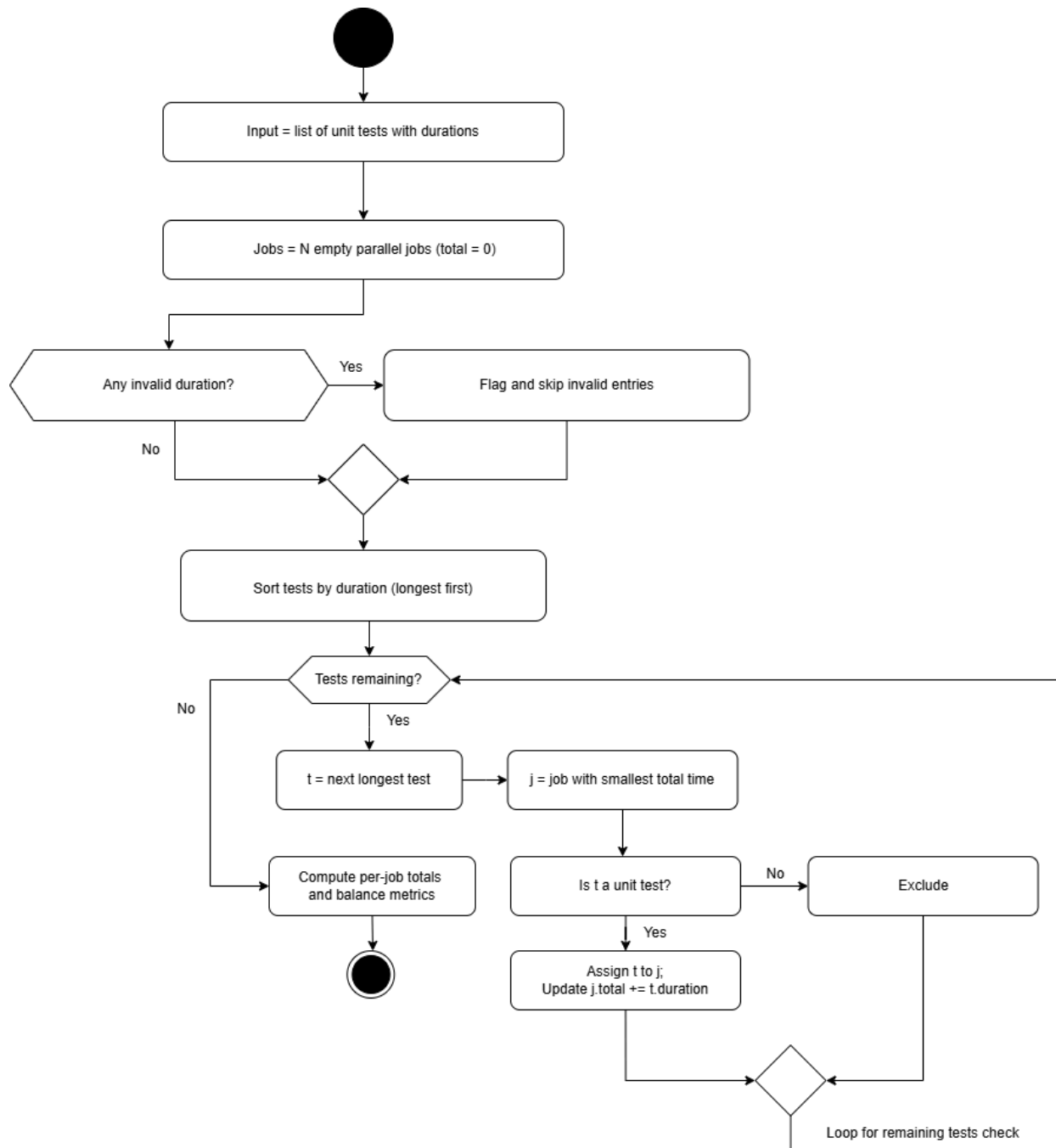


Figure 2: Test Allocation to Parallel Job Activity Diagram

3.4 Requirement 4 (FR4): CI Configuration Generation

Description:

The system must generate valid YAML configuration files for CI systems such as GitHub Actions and GitLab CI. These files define the job matrices and specify which test groups will execute within each job [21].

Criticality:

High: This enables direct integration of TestSplit’s optimisations into real CI workflows.

Technical Issues:

- Syntax and structure differences between CI platforms.
- Potential for invalid YAML causing failed builds.
- Platform-specific limits on job count or concurrency.

Mitigation Strategies:

- Maintain separate CI templates for each supported platform.
- Validate generated YAML files against the platform schema before saving.
- Provide a “dry-run” option to preview configurations before writing to disk.

Dependencies:

- Uses grouped test data from FR3.
- Requires consistent naming and structure to match CI runner expectations.
- Outputs configuration files that developers commit directly to their repositories.

3.5 Requirement 5 (FR5): Local Data Storage

Description:

TestSplit must persist profiling and grouping data locally to allow reuse between runs and enable trend analysis. This ensures users can view past performance metrics and verify improvements without re-profiling each time.

- Store test-level statistical data (mean duration, variance, instability flags).
- Persist environment metadata from each profiling run to support consistency checks.

Criticality:

High: Local persistence supports both usability and performance.

Technical Issues:

- Maintaining consistency between runs with different commits.
- Handling renamed or refactored tests.

- Preventing excessive disk-space growth over time.

Mitigation Strategies:

- Use lightweight file-based storage (JSON or SQLite) with indexing for faster lookups.
- Tag test records with commit hashes or UUIDs.
- Rotate or compress old profiling data automatically after a configurable number of runs.

Dependencies:

- Consumes profiling output from FR2 and grouping data from FR3.
- Feeds trend visualisation modules in the dashboard (FR7) and historical analysis (FR9).

3.6 Requirement 6 (FR6): Command-Line Interface (CLI)

Description:

A command-line interface must be provided for developers to run profiling, generate CI configurations, and view summary results. The CLI acts as the main control surface for integrating TestSplit into development workflows.

Criticality:

High: Primary user interaction point.

Technical Issues:

- Complex argument parsing and nested commands.
- Cross-platform compatibility across macOS, Linux, and Windows.
- Maintaining readability of terminal output.

Mitigation Strategies:

- Implement with Commander.js and chalk for structured output.
- Provide in-command help (--help) and usage examples.
- Use clear colour-coded status messages for success, warning, and error states.

Dependencies:

- Orchestrates FR1–FR5 in sequence.
- Reads/writes from local storage (FR5).
- Operates independently from the dashboard (FR7).

3.7 Requirement 7 (FR7): Local Dashboard for Historical Metrics

Description:

A local dashboard must display profiling metrics and job distribution results across multiple runs. Users can review total runtimes, test balances, and speed-up metrics through visual charts and tables [22], [23].

Criticality:

Medium: Not essential for functionality but critical for validation and demonstration.

Technical Issues:

- Efficiently rendering large datasets in the browser.
- Ensuring accurate visualisation of time-series metrics.
- Handling multiple datasets from different projects.

Mitigation Strategies:

- Implement React + Recharts or Chart.js with data virtualisation.
- Cache and paginate local datasets to reduce memory usage.
- Allow import/export of saved JSON profiles for reuse.

Dependencies:

- Uses profiling data from FR2 and historical storage from FR5.
- Displays grouping results from FR3 and configuration outcomes from FR4.

3.8 Requirement 8 (FR8): Containerised Execution Environment

Description:

The system must execute profiling and validation inside a Docker container to ensure consistent runtime conditions. This prevents hardware differences from skewing results and enables reproducible benchmarking [7], [8].

Criticality:

Medium: Not user-facing but essential for reliable performance evaluation.

Technical Issues:

- Managing container setup across operating systems.
- Balancing CPU and memory allocation to simulate realistic CI environments.
- Version drift between host and container images.

Mitigation Strategies:

- Provide a predefined Dockerfile and CLI alias to simplify setup.

- Use identical container configurations for baseline and optimised runs.
- Version-lock dependencies and image tags to maintain reproducibility.

Dependencies:

- Runs all profiling (FR2), grouping (FR3), and validation processes inside a stable environment.
- Supports consistent performance comparisons and future regression analysis (FR9).

3.9 Requirement 9 (FR9): Historical Trend Analysis

Description:

The system must analyse and visualise historical performance trends, allowing users to identify regressions, flaky tests, or newly introduced bottlenecks.

Extended Capabilities:

- Compute deltas in total suite duration, average duration, and per-test duration across runs.
- Compare balanceRatio and criticalPath over time to detect regressions in scheduling quality.
- Detect regressions in test performance by comparing current vs historical timing trends.

Criticality:

Medium: Supports long-term performance evaluation and reliability tracking.

Technical Issues:

- Mapping test identifiers across commits.
- Handling large amounts of time-series data.
- Requires consistent linking of test identifiers across commits for regression detection.
- Detecting statistically significant regressions.

Mitigation Strategies:

- Store incremental deltas rather than complete histories.
- Implement smoothing filters to highlight genuine anomalies [24].
- Allow users to manually tag or group related tests.

Dependencies:

- Built on profiling data (FR2) and local storage (FR5).
- Visualised through the dashboard (FR7).
- Runs on top of the containerised environment (FR8) for stable comparisons.

4. System Architecture

The TestSplit system uses a modular monorepo architecture that separates command-line operations, shared analysis logic, and data visualisation. The system consists of five primary modules: the CLI, Parsers, Algorithm, Generators, and Frontend.

These modules communicate via shared TypeScript models and local storage, enabling the system to operate effectively offline while remaining compatible with CI environments such as GitHub Actions and GitLab CI.

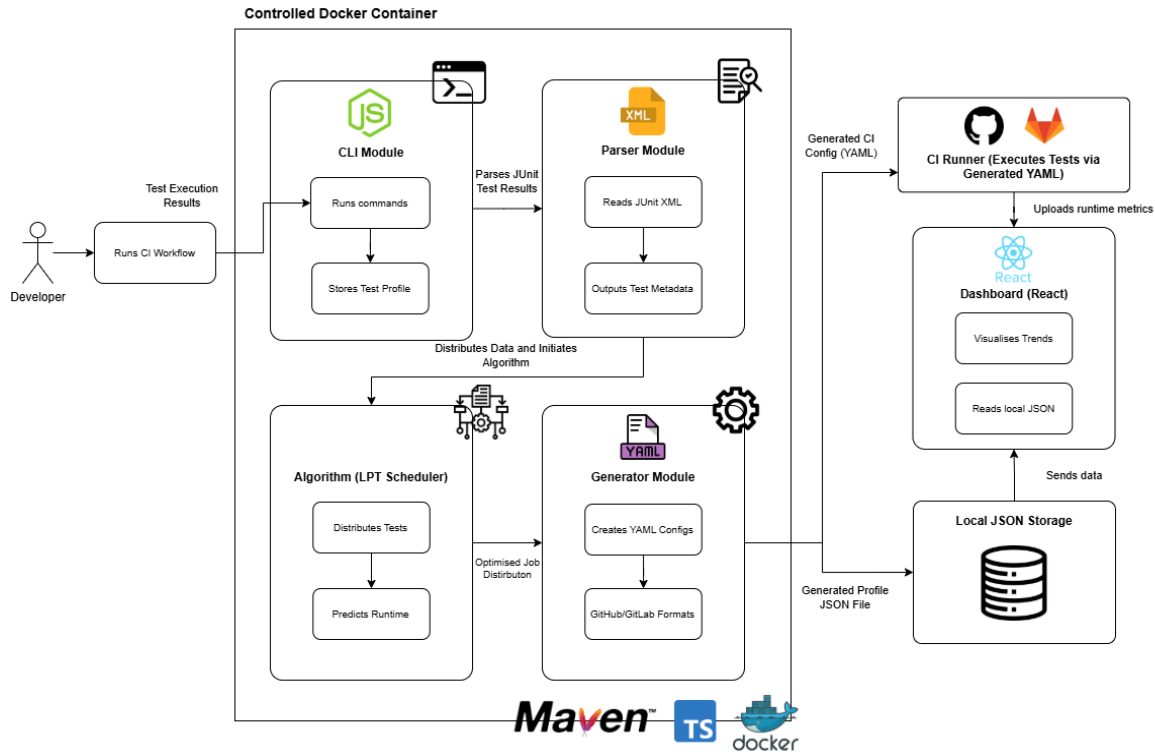


Figure 3: System Architecture Diagram

4.1 System Modules and Functions

Module: CLI (packages/cli/)

- Acts as the primary user interface for interacting with the system.
- Parses command-line arguments using Commander.js [25].
- Invokes the parser, algorithm, and generator modules to perform profiling, scheduling, and configuration generation.
- Formats terminal output as tables, JSON, or HTML.

- Manages local data persistence for profiling history.

Includes:

- commands/profile.ts Implements profiling functionality.
- commands/generate-config.ts Generates CI configuration files.
- commands/validate.ts Runs validation checks on generated configurations.
- utils/formatting.ts Handles terminal formatting and output presentation.
- storage/LocalStorage.ts Handles reading/writing persistent profiling data.

Module: Parsers (packages/shared/src/parsers/)

- Responsible for reading and interpreting **JUnit XML test result files** [26], [27].
- Extracts key metadata, including test name, duration, and status.
- Handles malformed or incomplete input gracefully to prevent analysis failures.

Includes:

- JUnitXMLParser.ts: Core XML parsing logic.
- TestParser.ts: Defines shared parser interfaces and data types.

Module: Algorithm (packages/shared/src/algorithm/)

- Implements the **Longest Processing Time (LPT)** scheduling algorithm for parallelisation.
- Calculates job distributions and predicted time savings.
- Generates metrics such as total duration, mean balance ratio and predicted speeds.

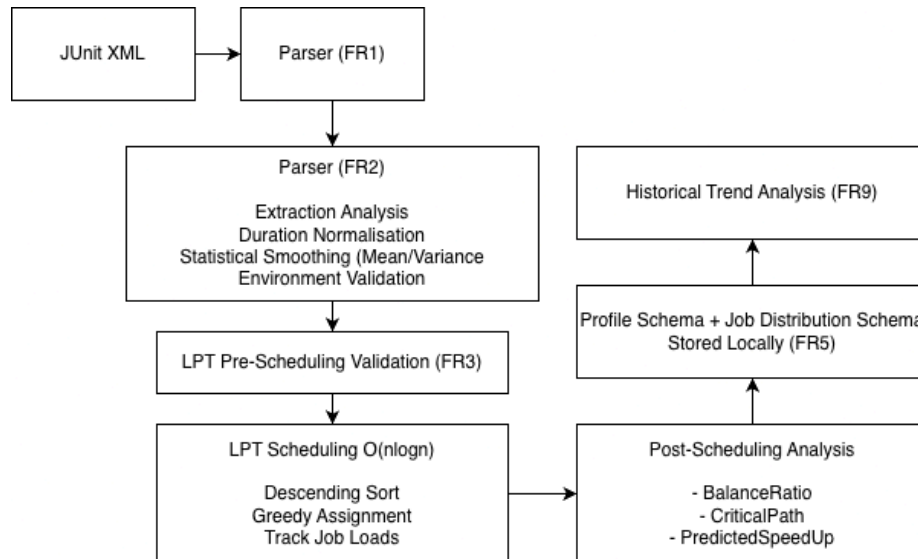


Figure 4: Profiler to Algorithm Flow

Includes:

- LPTScheduler.ts: Implements LPT scheduling logic ($O(n \log n)$ complexity).
 - Performs pre-scheduling validation (numeric durations, sorted order).
 - Implements greedy load-balancing by always assigning the next longest test to the job with the least accumulated time.
 - Computes post-scheduling metrics including balanceRatio, idealTime, and predictedSpeedUp.
 - Exposes distribution metrics for historical comparison across runs.
- statistics.ts: Performs statistical analysis and performance evaluation.
 - Computes multi-run smoothing (mean, variance, standard deviation) for each test.
 - Generates global metrics: suite-level mean, std deviation, and historical deltas.
- types.ts: Defines shared interfaces for test and job data structures.

Module: Generators (packages/shared/src/generators/)

- Creates valid **CI configuration files** in YAML format.
- Validates output syntax before saving to ensure compatibility with CI platforms.
- Supports generation for **GitHub Actions** and **GitLab CI**.

Includes:

- GitHubActionsGenerator.ts: GitHub-specific configuration.
- GitLabCIGenerator.ts: GitLab-specific configuration.
- ConfigGenerator.ts: Defines a shared interface for generators.

Module: Frontend (packages/frontend/)

- Provides a **local React dashboard** for visualising profiling and optimisation metrics.
- Displays performance trends, job balance charts, and execution-time distributions.
- Allows users to upload JSON results or view cached run data.

Includes:

- components/FileUpload.tsx: File upload and import handler.
- components/TrendChart.tsx: Visualises time-series and distribution trends.
- components/StatsSummary.tsx: Displays aggregated test statistics.
- pages/Dashboard.tsx: Main entry page for viewing analysis results.

4.2 Interaction Flow (High-Level)

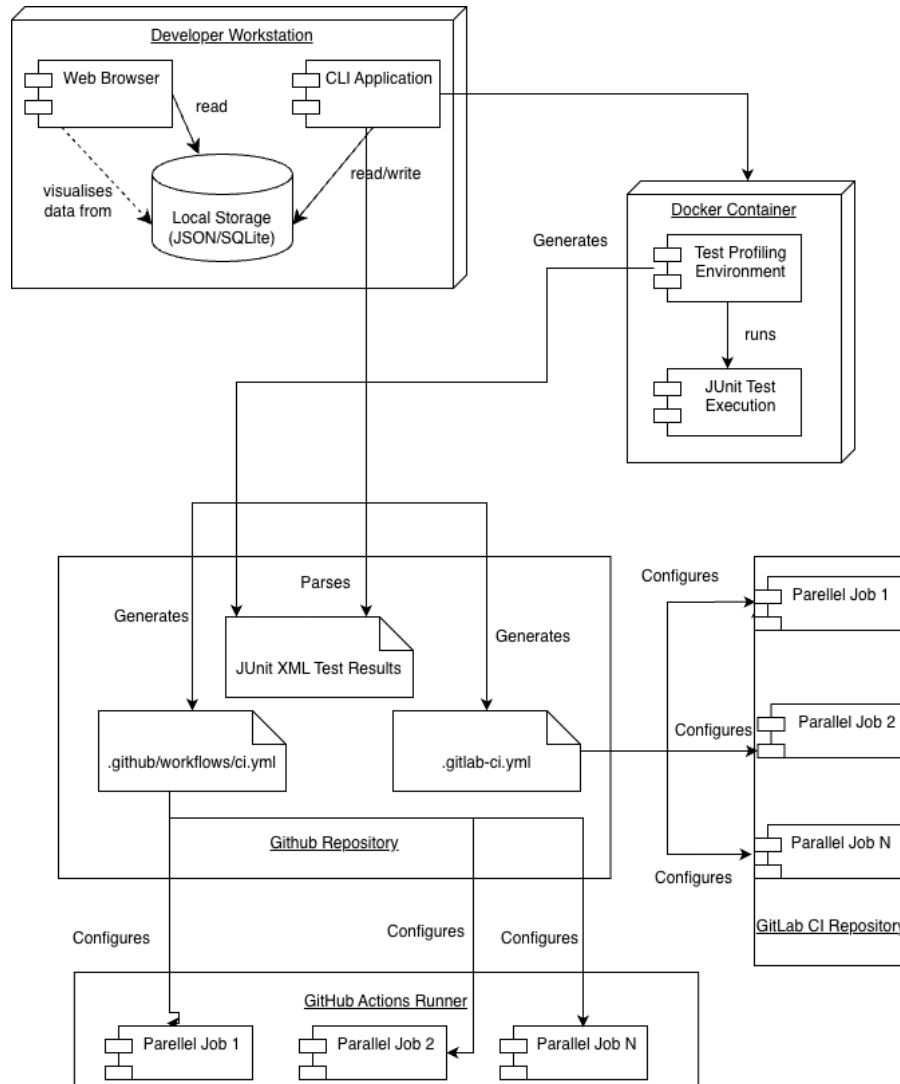


Figure 5: Deployment Diagram

Flow 1: Profile Test Suite

1. The user runs the TestSplit profile from the CLI.
2. The **Parsers** module reads JUnit XML test results from the project's target/surefire-reports/ directory.
3. Extracted test durations and metadata are passed to the **Algorithm** module.
4. The **LPT Scheduler** determines the optimal distribution of jobs across a configurable number of jobs.
5. The CLI stores profiling data locally (e.g., ~/.testsplit/history.json).
6. Results are displayed in the terminal and optionally exported for dashboard analysis.

Flow 2: Generate CI Configuration

1. The user runs `testsplit generate-config --platform github --jobs 4`.
2. The **Algorithm** output is passed to the **Generators** module.
3. A valid YAML configuration file is produced and validated.
4. The configuration is written to `.github/workflows/ci.yml` or equivalent for GitLab.

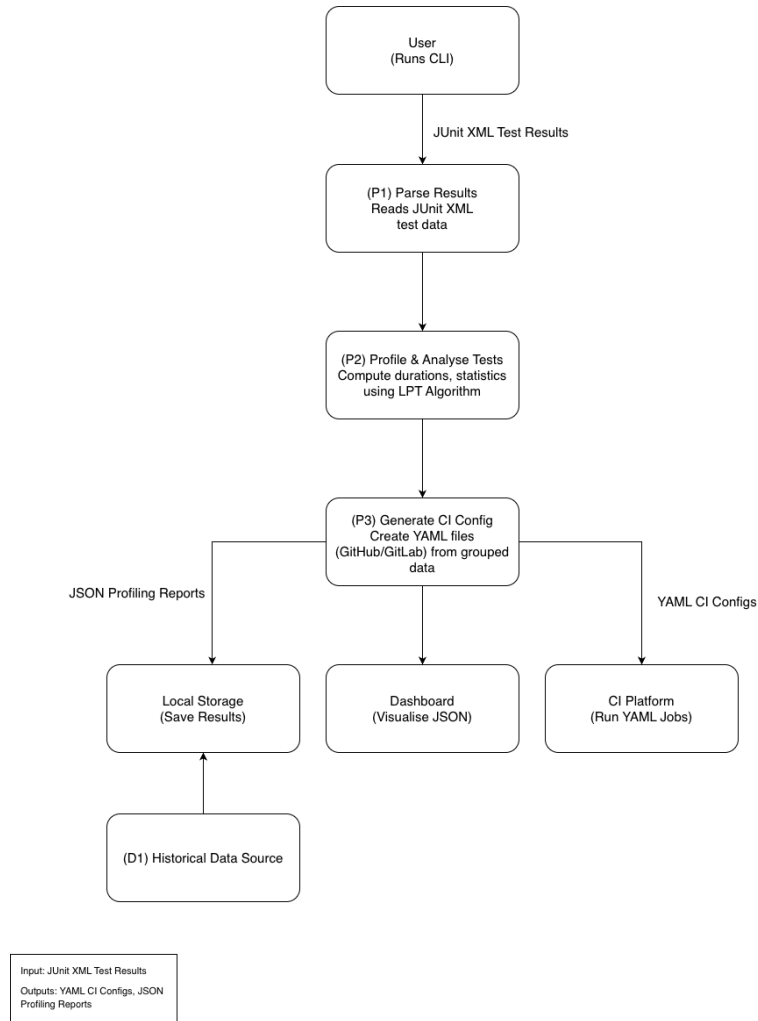


Figure 6: Data Flow Diagram

Additional flows for visualisation and validation are illustrated in Section 4.4 with sequence diagrams.

4.3 Environmental Context

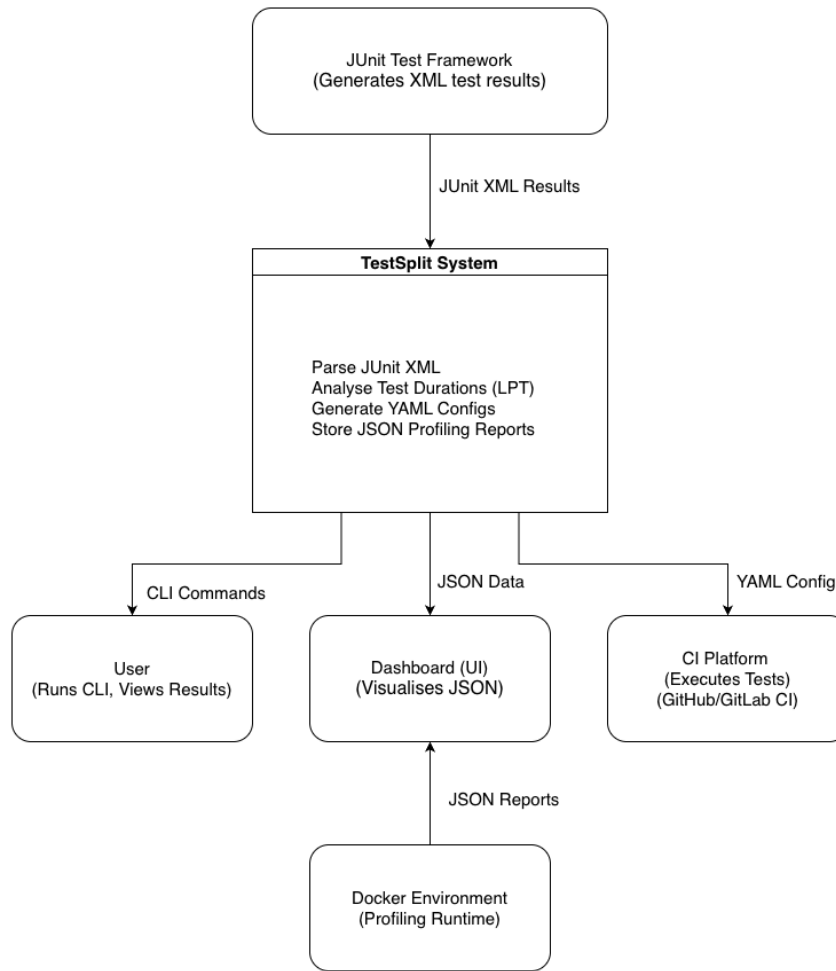


Figure 7: System Context Diagram

Docker Environment:

- All profiling, parsing, and validation are executed inside a Docker container for reproducibility.
- Ensures consistent resource allocation (e.g., CPU cores, memory) and eliminates hardware bias [28].
- Defines a uniform baseline for comparing single-threaded vs. multi-threaded CI job scheduling.

CLI Environment:

- Runs locally on developer workstations (macOS, Linux, or Windows).
- Requires Node.js 18+ and npm 8+.
- Reads/writes profiling data from the local filesystem.
- Interfaces with the Docker container for stable, reproducible runs.

CI Environment (Generated Configurations):

- Executes on **GitHub Actions** or **GitLab CI** runners.
- Requires test frameworks supporting sharding or filtering.
- Uses standard runner permissions for dependency installation and test execution.
- Generated configurations can be validated in the same containerised environment for accuracy.

Dashboard Environment:

- Runs locally within a modern web browser (Chrome, Firefox, Edge, Safari).
- No network access required.
- Accessed via a local development server started by the CLI (e.g., TestSplit dashboard).
- Visualises historical profiling data and performance trends.

This structure ensures a reproducible, containerised environment for profiling and testing while maintaining developer accessibility. The CLI performs all profiling and configuration generation, the container ensures consistency across environments, and the dashboard enables intuitive performance analysis.

4.4 Reference Diagrams

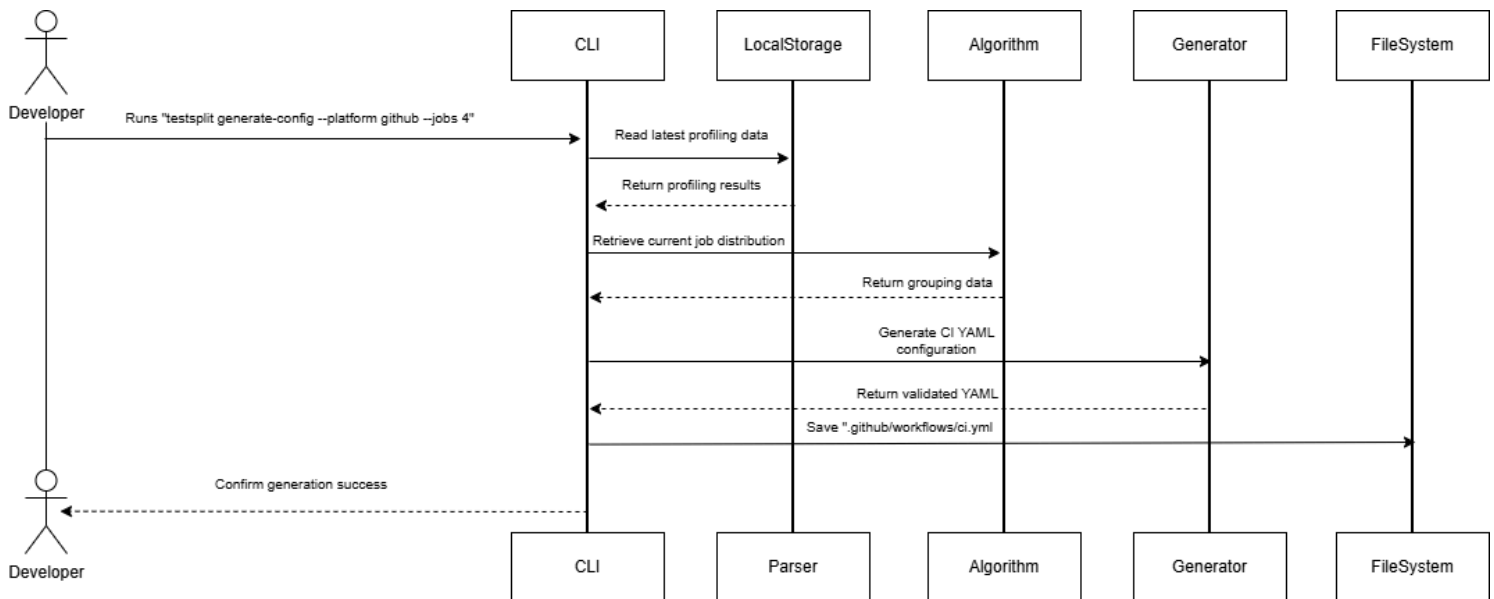


Figure 8: Sequence Diagram - Configuration Generation

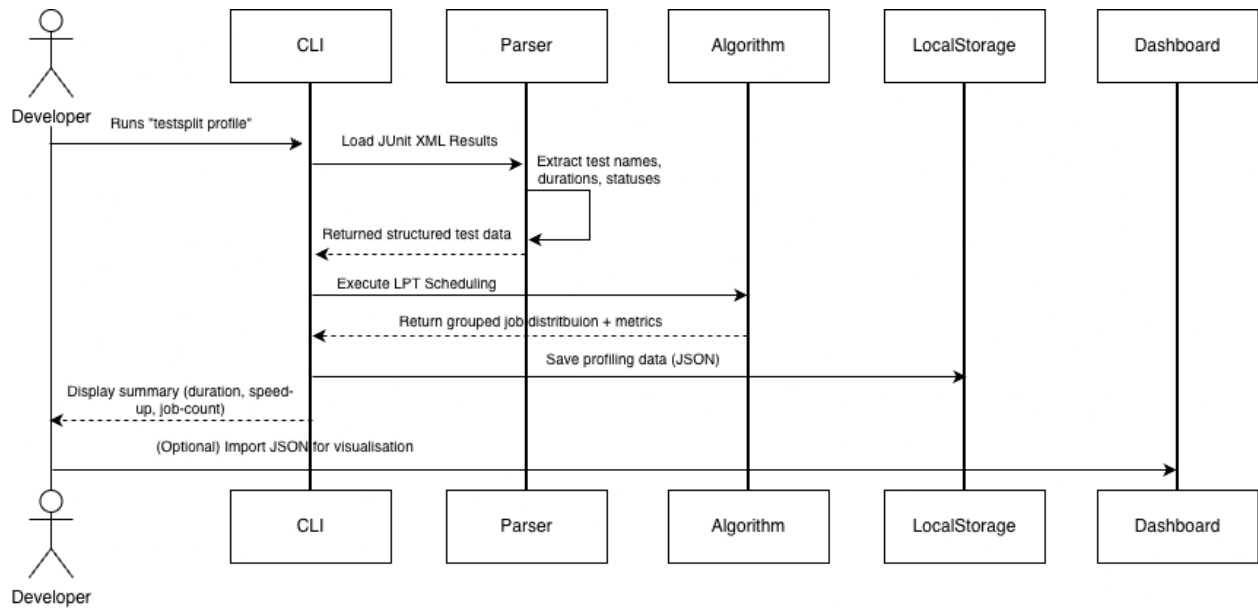


Figure 9: Sequence Diagram - Dashboard Visualisation

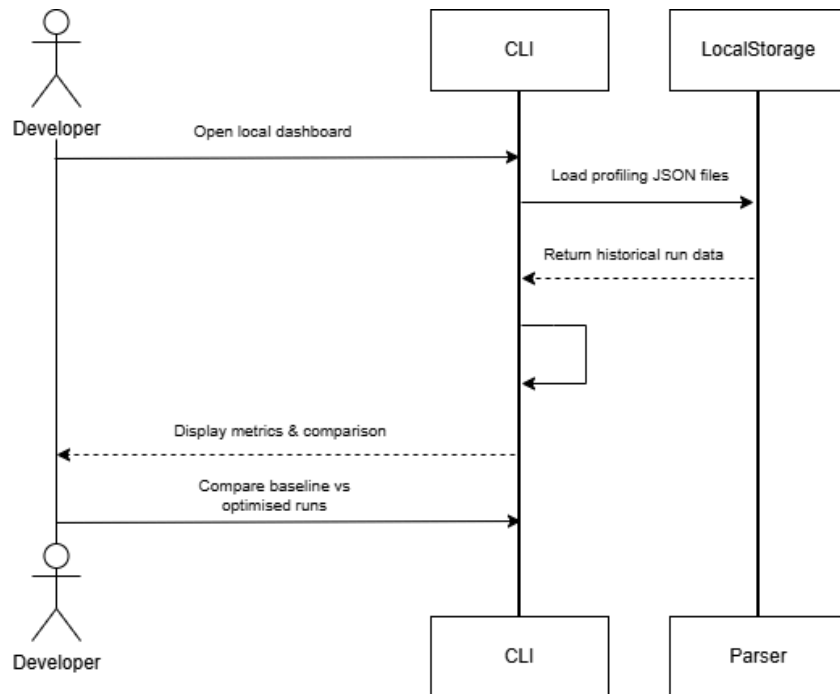


Figure 10: Sequence Diagram - Run Comparison

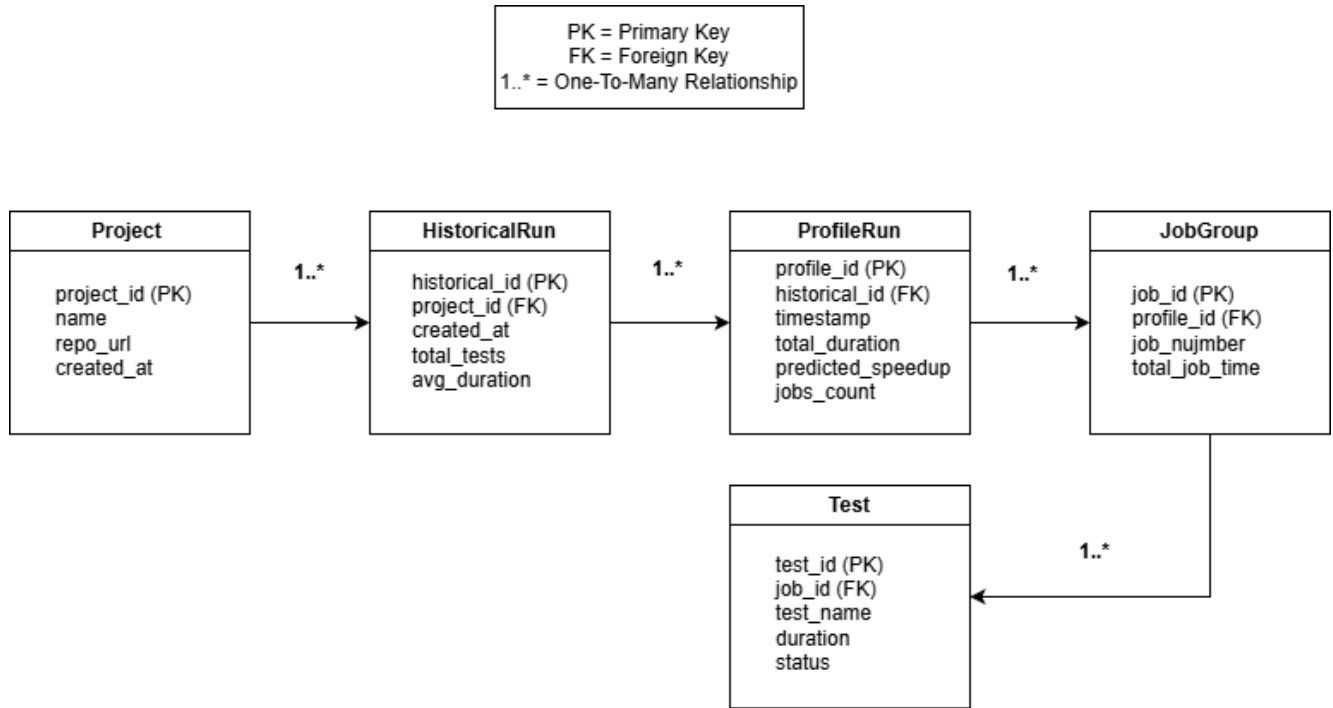


Figure 11: ER Diagram

4.4.1 Job Distribution Schema

The Job Distribution Schema defines how individual tests are grouped into parallel execution jobs following the Longest Processing Time (LPT) scheduling algorithm. It represents the output of the LPTScheduler module and is used by the ConfigGenerator to produce platform-specific YAML configurations for Continuous Integration (CI) systems.

Each job group contains the set of tests assigned to it, the total execution duration, and the balance metrics used to evaluate the distribution's efficiency.

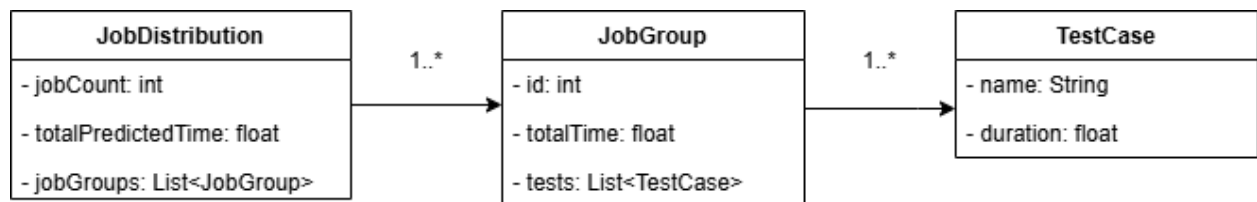


Figure 12: Job Distribution Schema Diagram

Usage: This schema is written to disk by the LocalStorage module, consumed by the ConfigGenerator, and visualised in the Dashboard as part of the profiling output.

4.4.2 Profile Schema

The Profile Schema represents the data produced during a single profiling run. It captures individual test results, aggregated metrics, and environment metadata, forming the foundation for later analysis and CI optimisation.

This schema is generated by the Profiler module and stored locally in JSON format.

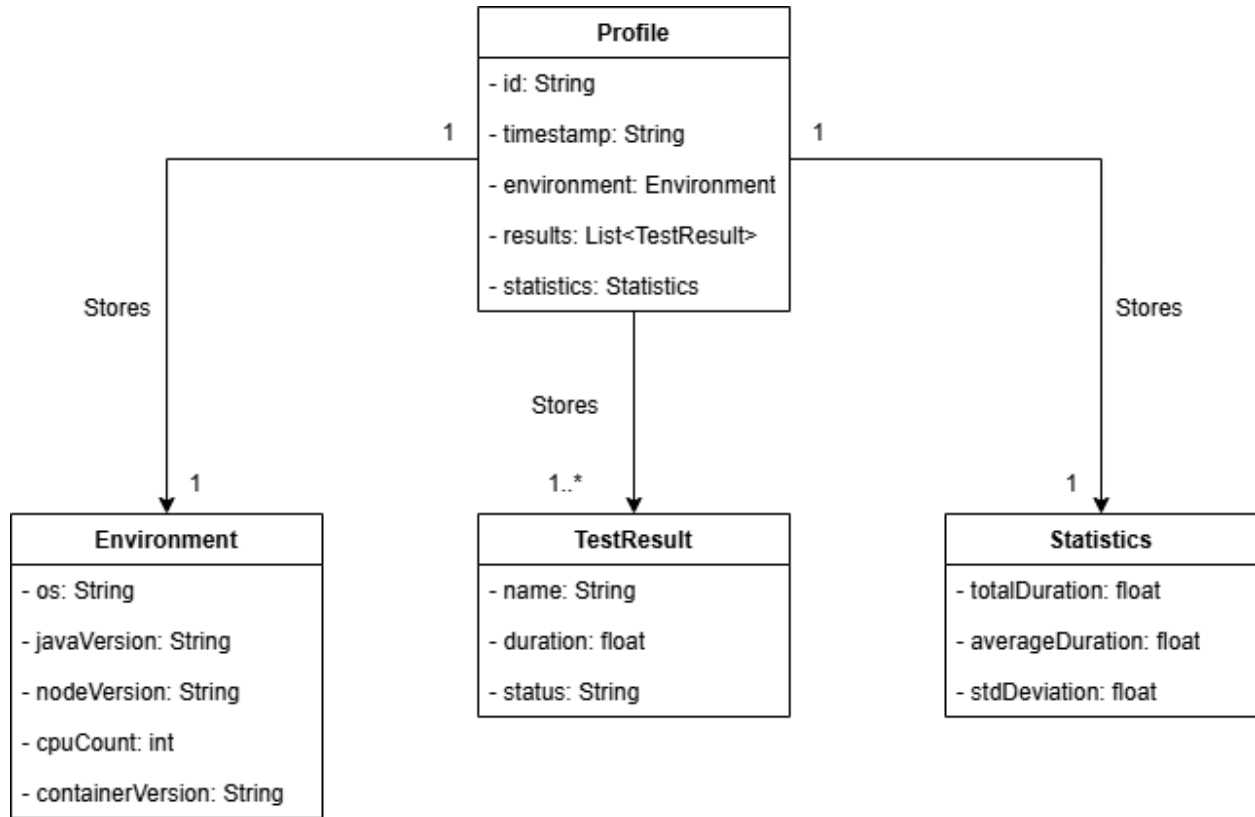


Figure 13: Profile Schema Diagram

Usage: The Profile Schema is the core input for:

- The LPTScheduler, which computes optimal test grouping.
- The Dashboard, which visualises results through charts and summaries
- The Historical Data Schema which aggregates multiple profiles for regression analysis.

4.4.3 Historical Data Schema

The Historical Data Schema maintains an aggregated record of multiple profiling sessions to enable long-term performance tracking and regression detection. It forms the basis for trend visualisation and comparison in the Dashboard module.

Each historical record references one or more Profile objects, preserving data continuity across commits, profiling sessions, and configuration updates.

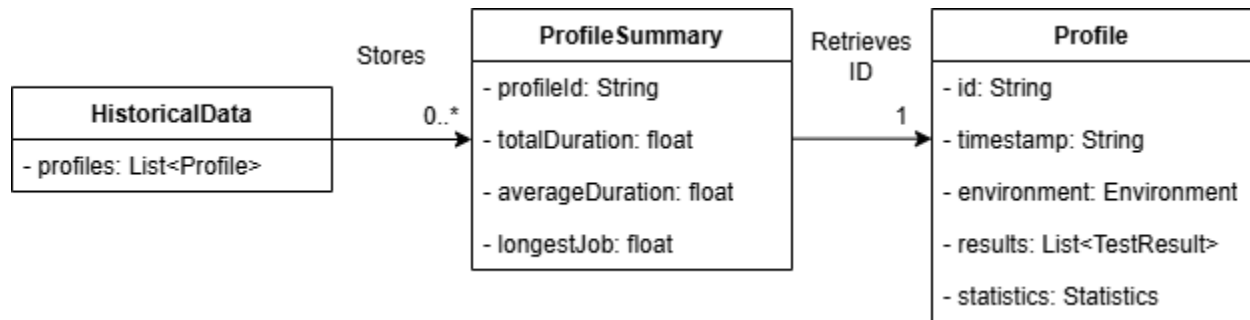


Figure 14: Historical Data Schema Diagram

Usage: This schema is used exclusively by the Dashboard to plot performance trends and identify regressions over time.

These three components describe the overall class diagram shown below, detailing how the objects in the system interact.

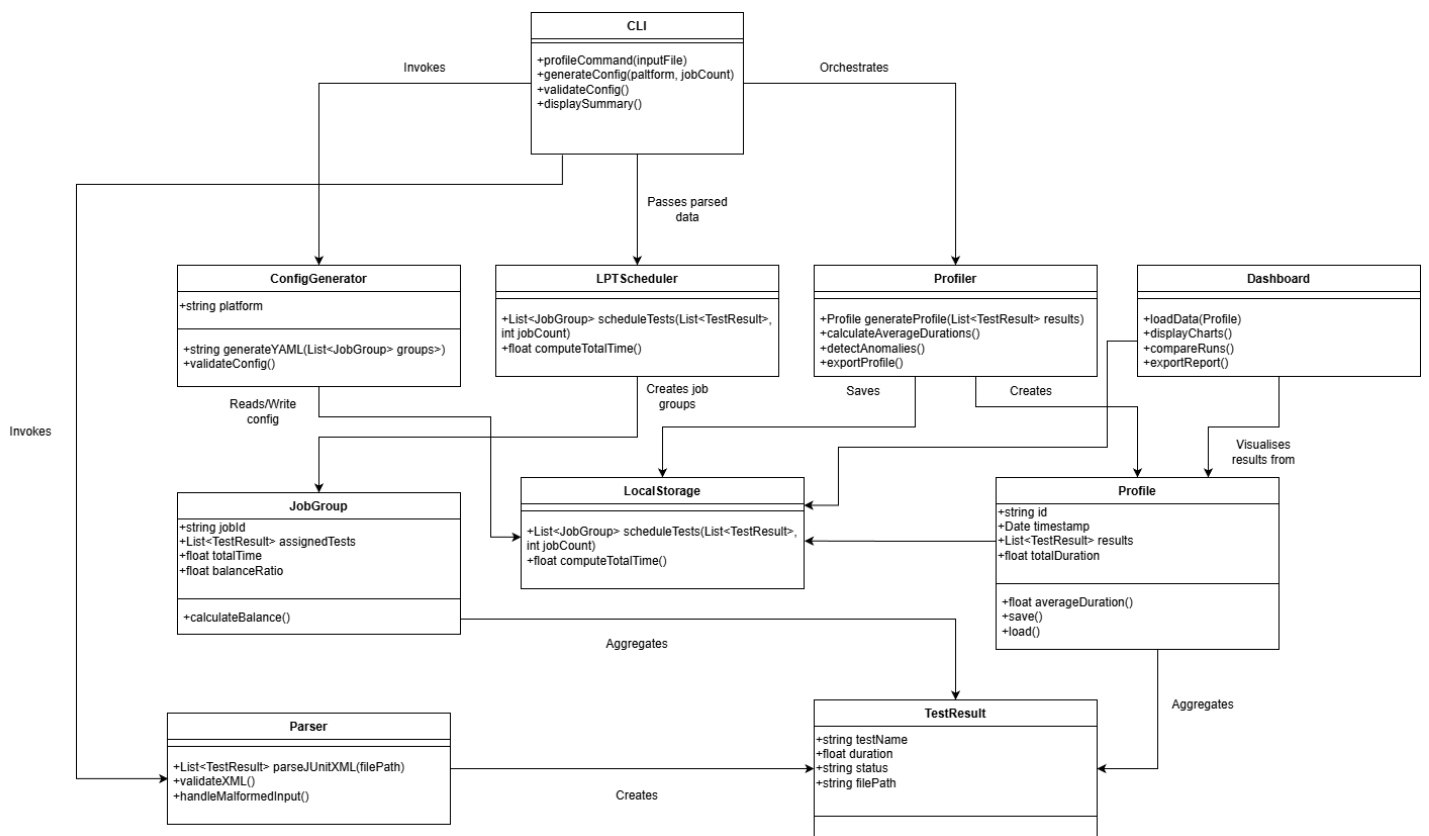


Figure 15: High-Level Class Diagram

4.5 Non-Functional Design Choices

Scalability:

- Runs as a local command for one user at a time.
- Safe to run in CI: each run uses its own temp folder, so jobs don't clash.

Performance:

- Reads JUnit XML in a streaming way so large files don't exhaust memory.
- Scheduling happens in memory and completes quickly for typical project sizes.

Reliability:

- Validates config files before saving.
- If saving history fails, the run still completes and shows a clear warning.
- Can run inside Docker to keep runs consistent across machines.

Security:

- Works offline by default; no data is sent anywhere.
- Saves files with safe permissions.
- Rejects unsafe file paths and broken or suspicious XML.

Usability:

- Helpful --help with examples.
- Clear error messages with next steps.

Maintainability:

- TypeScript with linting.
- CI runs checks on every commit.

Portability:

- Works on Linux, macOS, and Windows.
- Supports the current versions of Chrome, Firefox, Safari, and Edge.
- A Docker image is available for those who prefer containerised runs.

4.6 Non-Functional Requirements

4.6.1 Runtime Environment:

- Profiling must run inside a Linux-based Docker container.
- The container must use fixed versions of the OS, software tools and build utilities.

- Project dependencies must stay the same while profiling results are being collected.
- External updates must not occur during profiling as they may change test results.

4.6.2 Test Suitability Requirements

- The system supports unit tests that run independently and do not rely on other tests.
- Repositories used for evaluation must contain only unit tests.
- Tests that require a fixed order/multi-step workflows are not supported.

4.6.3 Quality Requirements

- Statement coverage must be used to assess how much of the code is exercised during testing, with any other decision points in the code being covered if necessary.

4.6.4 Performance Requirements

- The system must remain practical to use for both small and large sets of independent unit tests.
- No fixed timing targets are identified, because performance is highly dependent on the user's hardware.

5. Testing and Evaluation Plan

5.1 Testing Goals

- **Correctness:** The LPT scheduler yields a valid, non-overlapping partition of tests into parallel jobs.
- **Accuracy:** The system provides a prediction of parallel job completion time and records actuals for later comparison.
- **Robustness:** The system handles malformed/partial JUnit XML and large unit-test suites without crashing, instead emitting actionable errors.
- **Usability:** CLI and UI flows are discoverable; error and help texts are explicit.

5.2 Test Types and Mapping

Unit Tests (packages/*/src/**/__tests__/*_test.ts)

- **Parser:** JUnitXMLParser.test.ts – valid XML, missing attributes, malformed structure.
- **Algorithm:** LPTScheduler.test.ts – partition invariants and edge cases.
- **Statistics:** statistics.test.ts – descriptive metrics (mean, median, p95).
- **Generators:** GitHubActionsGenerator.test.ts, GitLabCIGenerator.test.ts – YAML shape/schema validation.
- **CLI:** profile.test.ts (args/help/errors); generate-config.test.ts (file output).

Property-Based Tests (packages/shared/src/algorithm/__tests__/*_property.test.ts)

- **Partition invariant:** Each test identifier appears exactly once across parallel jobs.
- **Conservation:** Sum of parallel job durations equals total duration within parser precision.
- **Monotonicity:** Check that the increase in available jobs does not produce a worse total predicted time.
- **Input validation:** Non-numeric/negative durations rejected with clear errors.

Integration Tests (tests/integration/*.test.ts)

- profile-flow.test.ts – parse, profile, store round-trip.
- generate-config-flow.test.ts – CI config generation and schema validation.
- historical-storage.test.ts – read/write consistency and rotation.

E2E Tests (using Playwright in tests/e2e/*.spec.ts)

- upload-file.spec.ts – upload JSON and render charts.
- trend-visualization.spec.ts – chart interaction and export.

Validation Tests (docs/validation-results.md)

- Produce a report comparing baseline vs. optimised runs; include methodology and raw observations, without pass/fail thresholds.

5.3 Evaluation Criteria and Thresholds

- **Scheduler correctness:** Each test is assigned to exactly one parallel job (one worker running in parallel). No duplicates. The output files can be saved and loaded back.
- **Predictions vs. real runs:** We record our predictions for each parallel job and what happened, and show them side-by-side for review.
- **Build and code health:** The project compiles, type-checks, and lints cleanly. Unit tests cover important parts.
- **Developer and user experience:** Core commands (profile, generate-config) are easy to find with --help. Error messages are specific and helpful.
- **Repeatable results:** With the same inputs and environment, results are the same each time.
- **Robustness:** Bad or very large JUnit XML files are handled without crashes, with actionable errors.
- **Documentation:** A concise README, quick start, and CLI reference live at the repository root.

5.4 Test Data

- **Open-Source Repositories:** Repositories with pure unit tests not used in integration tests will be selected as test data. This includes:
 - **apache/commons-lang:** src/test/java/org/apache/commons/lang3/
 - **jhy/jsoup:** src/test/java/org/jsoup/
 - **google/truth:** core/src/test/java/com/google/common/truth/

- **google/guava:** guava-tests/test/com/google/common/base/
- **Example Applications (examples/apps/):** Small projects used for demonstrations and validation inside the Docker environment.

Only repositories with genuine unit tests are used for evaluation. Projects that mainly contain integration tests are not suitable for this tool and are not included in testing.

6. Preliminary Schedule

TestSplit Project Timeline

CSC1097 Final Year Project | December 2025 - April 2026
Authors: Samson Oloruntola & Marjia Siddik | Supervisor: Dr Paul Clarke

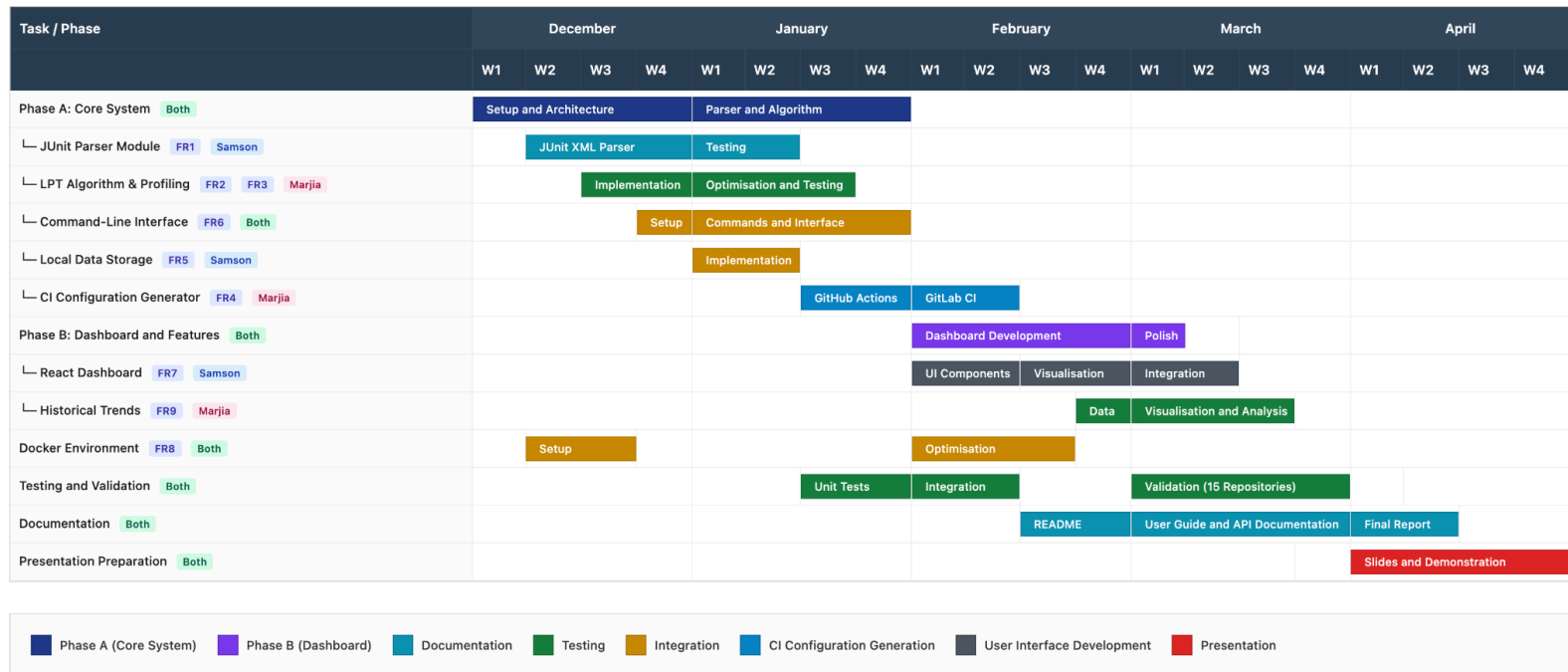


Figure 16: GANTT Chart

7. Appendices

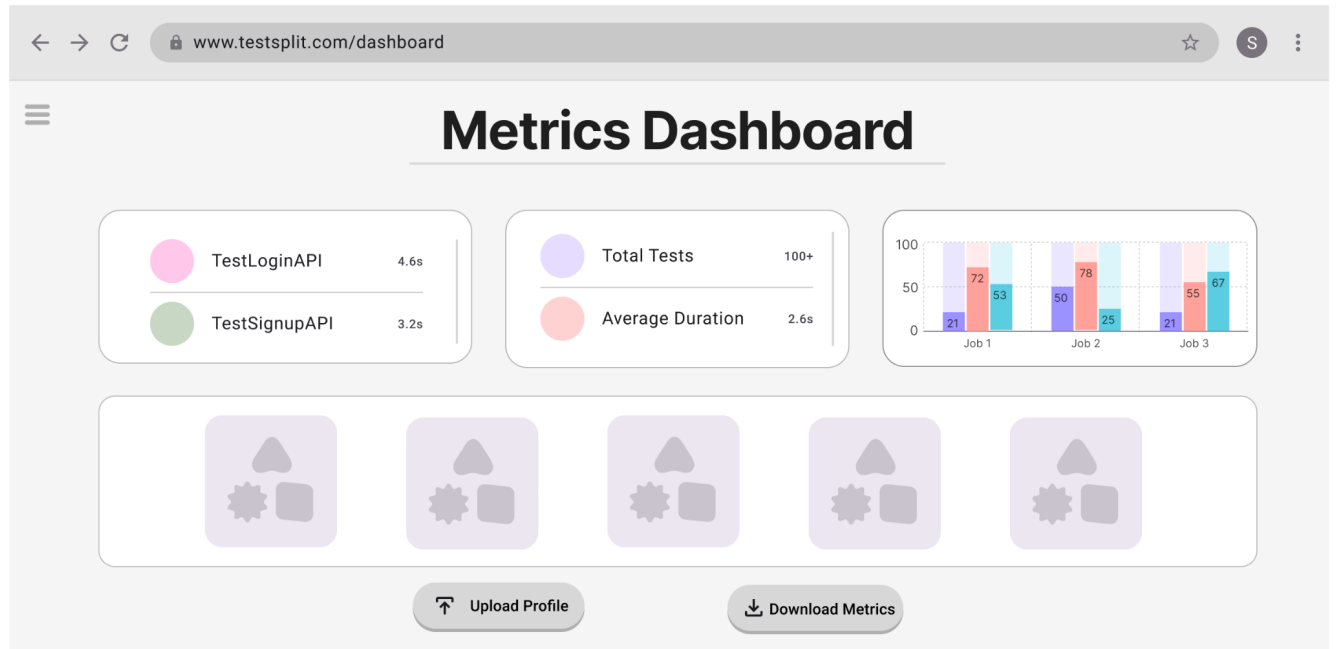


Figure 17: Wireframe for Static Dashboard

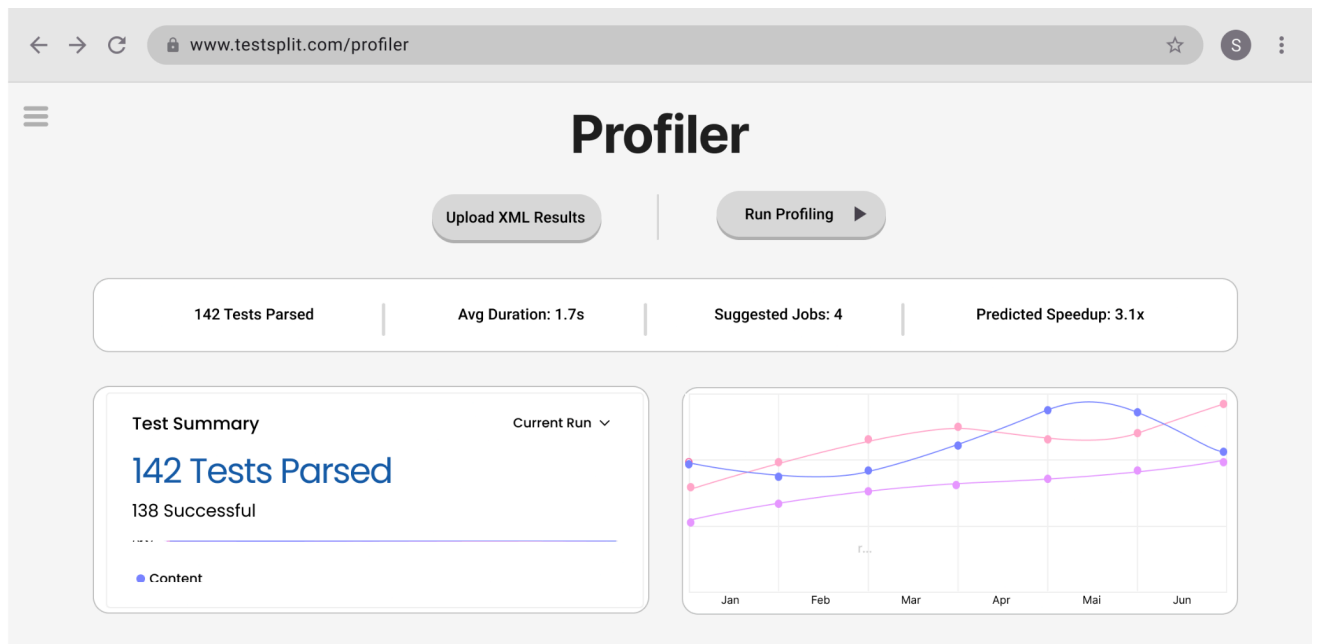


Figure 18: Wireframe for Profiler

8. References

- [1] R. L. Graham, “Bounds on multiprocessing timing anomalies,” *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, 1969.
- [2] X. Xiao, “A direct proof of the $4/3$ bound of LPT scheduling rule,” *Atlantis Press*, 2017.
- [3] F. Della Croce and R. Scatamacchia, “The longest processing time rule for identical parallel machines,” *Journal of Scheduling*, vol. 23, pp. 523–532, 2020.
- [4] W. Felidré, L. Furtado, D. A. da Costa, B. Cartaxo and G. Pinto, “Continuous Integration Theater,” *CoRR*, vol. abs/1907.01602, 2019.
- [5] O. Martin, “Unlocking the true potential of CI/CD pipelines for improved productivity,” *Calibo Blog*, 26 Jul. 2024. Available: <https://calibo.com/blog/ci-cd-pipelines-for-improved-productivity/>
- [6] “Four of the biggest challenges teams face with CI/CD,” *Stride.Build Blog*, 17 Jan. 2023. Available: <https://www.stride.build/blog/four-of-the-biggest-challenges-teams-face-with-ci-cd/>
- [7] Docker Documentation, *Manage Compute Resources for Containers*, Docker Inc., 2025. [Online]. Available: https://docs.docker.com/engine/containers/resource_constraints/
- [8] Turnbull, J., *The Docker Book*, James Turnbull, 2014.
- [9] GitHub Actions Documentation, *Choosing the Runner for a Job*, GitHub, 2025. [Online]. Available: <https://docs.github.com/en/actions/how-tos/write-workflows/choose-where-workflows-run/choose-the-runner-for-a-job>
- [10] GitLab Documentation, *GitLab CI/CD Pipeline Configuration Reference*, GitLab, 2025.
- [11] Boettiger, C., “An Introduction to Docker for Reproducible Research,” *ACM SIGOPS Operating Systems Review*, 2015.
- [12] Apache Maven Project, *Maven Surefire Plugin – Generating JUnit XML Reports*, The Apache Software Foundation, 2025. [Online]. Available: <https://maven.apache.org/surefire/maven-surefire-plugin/>
- [13] JUnit.org, *JUnit 5 User Guide – Running Tests and XML Output Format*, JUnit Team, 2025. [Online]. Available: <https://junit.org/junit5/docs/current/user-guide/>
- [14] SonarSource Community, *Large JUnit XML Files and Analysis Performance Issues*, SonarSource, 2024. [Online]. Available: <https://community.sonarsource.com/>
- [15] Massol, V., & Husted, T., *JUnit in Action*, Manning Publications, 2003.

- [16] Grune, D., & Jacobs, C., *Parsing Techniques: A Practical Guide*, Springer, 2008.
- [17] Zakhour, S., et al., *Java Performance: The Definitive Guide*, Oracle Press, 2014.
- [18] Amdahl, G., “Validity of the Single Processor Approach to Achieving Large-Scale Computing Capabilities,” *AFIPS Conference Proceedings*, 1967.
- [19] Herlihy, M., & Shavit, N., *The Art of Multiprocessor Programming*, Morgan Kaufmann, 2012.
- [20] M. Dell’Amico, M. Iori, S. Martello, and M. Monaci, “Scheduling with inexact job sizes: The merits of shortest processing time first,” *arXiv preprint*, arXiv:1907.04824, 2019.
- [21] Ben-Kiki, O., Evans, C., & Ingerson, B., *YAML Ain’t Markup Language (YAML™) Version 1.2 Specification*, 2009.
- [22] Shneiderman, B., Plaisant, C., Cohen, M., & Jacobs, S., *Designing the User Interface*, Pearson, 2017.
- [23] Few, S., *Information Dashboard Design*, Analytics Press, 2006.
- [24] Hand, D., Mannila, H., & Smyth, P., *Principles of Data Mining*, MIT Press, 2001.
- [25] Node.js Foundation, *Worker Threads and Parallelism in Node.js*, Node.js Foundation, 2025.
[Online]. Available: https://nodejs.org/api/worker_threads.html
- [26] Pinedo, M., *Scheduling: Theory, Algorithms, and Systems*. Springer, 2016.
- [27] Duvall, P., Matyas, S., & Glover, A., *Continuous Integration: Improving Software Quality and Reducing Risk*. Addison-Wesley, 2007.
- [28] Silberschatz, A., Galvin, P. B., & Gagne, G., *Operating System Concepts* (10th ed.), Wiley, 2018.